

Plant Disease Detection Using Deep Transfer Learning

Abstract

Plant diseases are responsible for significant agricultural losses worldwide, with estimates suggesting up to 40% of global crop production is lost annually due to pests and pathogens. Early and accurate identification of disease is critical for timely intervention, yet manual diagnosis by trained agronomists is costly, slow, and unavailable in many rural regions. This project presents a deep learning system for automated plant disease classification using transfer learning on a fine-tuned ResNet50 convolutional neural network. The model is trained on the PlantVillage dataset — 87,867 leaf photographs spanning 38 disease and healthy categories across 14 plant species. The convolutional backbone is initialised with ImageNet weights, and only the final classification layer is retrained to adapt the pretrained visual representations to the domain of plant pathology. A web application built with Streamlit provides single-image diagnosis and batch processing with CSV export, accompanied by a curated disease knowledge base that returns treatment and prevention guidance alongside each prediction. The system achieves 89.82% overall accuracy on the held-out validation split of 17,572 images using backbone-frozen transfer learning, with a macro F1-score of 89.51%, demonstrating that a frozen ImageNet backbone provides a strong foundation for fine-grained plant disease recognition.

1. Introduction

1.1 Background

Agriculture is the backbone of food security globally, yet crop diseases consistently threaten yields across all major staple and cash crops. The Food and Agriculture Organization of the United Nations estimates that plant diseases and pests cause annual losses of approximately USD 220 billion worldwide. Timely identification of the specific pathogen or stress condition affecting a crop is the prerequisite for any effective treatment. Traditional diagnosis relies on expert agronomists conducting visual field inspections — a process that is geographically limited, expensive, and inherently subjective.

1.2 Motivation

The widespread availability of smartphone cameras and mobile internet access means that farmers in remote areas now possess high-quality image capture devices. A computer vision system capable of classifying a leaf photograph in under one second can serve as an on-demand diagnostic tool at the point of need, requiring no specialist present. Deep convolutional neural networks have demonstrated strong performance on fine-grained visual recognition benchmarks and are a natural fit for this problem.

1.3 Problem Statement

Given a photograph of a plant leaf, automatically classify the leaf into one of 38 categories corresponding to disease-free specimens or specific disease conditions across 14 plant species, and provide the user with practical treatment and prevention guidance for the predicted condition.

1.4 Objectives

- Fine-tune a pretrained ResNet50 model to classify 38 plant disease categories with high accuracy.
- Build an accessible web application supporting single-image and batch-image diagnosis.
- Provide per-prediction disease information drawn from a structured knowledge base.
- Design the system to run on CPU-only hardware, making it deployable without specialist infrastructure.
- Evaluate performance rigorously with precision, recall, F1-score, and a confusion matrix.

2. Literature Review

2.1 PlantVillage and Early Deep Learning Approaches

Mohanty, Hughes, and Salathé (2016) introduced the PlantVillage dataset and demonstrated that deep convolutional neural networks — specifically AlexNet and GoogLeNet — could achieve up to 99.35% accuracy on controlled-background leaf images. Their work established the benchmark that subsequent studies measure against and highlighted the key limitation that performance drops significantly on field-condition photographs, where backgrounds are complex and lighting is inconsistent.

2.2 Residual Networks

He, Zhang, Ren, and Sun (2016) introduced Deep Residual Learning, demonstrating that networks with skip connections — which add the block input directly to its output — could be trained to 152 layers without degradation. ResNet50 emerged as the practical sweet spot: deep enough to capture high-level disease-specific features, shallow enough for CPU-viable inference. The residual formulation directly solves the vanishing gradient problem that limited plain deep networks and remains widely adopted in transfer learning pipelines.

2.3 Transfer Learning for Plant Pathology

Ferentinos (2018) conducted a systematic evaluation of deep learning architectures on the PlantVillage dataset and found that VGG and AlexNet architectures, when fine-tuned from ImageNet, outperformed models trained from scratch — confirming the value of transfer learning even when the source and target domains differ substantially. Barbedo (2016) reviewed classical computer vision approaches (colour histogram analysis, texture features, SVM classifiers) and concluded that deep learning supersedes them on both accuracy and generalisability.

2.4 Optimisation

Kingma and Ba (2015) introduced Adam, an adaptive-learning-rate optimiser that maintains per-parameter first and second moment estimates of the gradient. Adam has become the standard choice for fine-tuning pretrained CNNs because its adaptive step sizes naturally handle the heterogeneous gradient scales across layer groups that arise when only some layers are unfrozen.

2.5 Comparison of Approaches

Method	Accuracy	Limitation
AlexNet (Mohanty et al., 2016)	99.35%	Controlled images only; overfits to lab conditions
GoogLeNet (Mohanty et al., 2016)	98.90%	High computational cost; complex architecture

VGG16 (Ferentinos, 2018)	~97%	138M parameters; slow inference; no skip connections
SVM on hand-crafted features (Barbedo, 2016)	~80–85%	Manual feature engineering; does not generalise
ResNet50 backbone-frozen (this project)	89.82%	Lab images; backbone-frozen mode

3. Dataset

3.1 Description and Source

This project uses the New Plant Diseases Dataset (PlantVillage), publicly available on Kaggle (<https://www.kaggle.com/datasets/vip00000l/new-plant-diseases-dataset>). The dataset originates from the PlantVillage project at Penn State University, which systematically collected leaf photographs under controlled laboratory conditions with uniform grey or black backgrounds to isolate the leaf from environmental noise.

3.2 Statistics

Split	Images	Classes	Plants
Training	~70,295	38	14
Validation	~17,572	38	14
Total	~87,867	38	14

Classes are approximately balanced across disease variants and healthy specimens. Each class contains between 400 and 2,000 images. The 38 categories span conditions including fungal infections, bacterial spots, viral diseases, and nutrient deficiencies across species such as tomato, potato, apple, corn, and grape.

3.3 Preprocessing Steps

1. Resize to 224×224 pixels — the input size expected by ResNet50's global average-pool head.
2. Convert to RGB — all images are loaded as PIL.Image in RGB mode, normalising RGBA, palette, and greyscale inputs to a consistent 3-channel format.
3. ImageNet normalisation — each channel is normalised to zero mean and unit variance using the ImageNet population statistics: mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]. The same statistics are used at training and inference time to match the distribution seen during ImageNet pretraining.
4. Training augmentation (training split only) — random horizontal flip ($p = 0.5$) and random rotation in $[-15^\circ, +15^\circ]$ are applied to improve generalisation without introducing distribution mismatch.

4. Methodology

4.1 System Architecture

The system is structured as a modular pipeline with three logical layers: an input layer (image validation and preprocessing), a model layer (ResNet50 inference), and an output layer (confidence thresholding, knowledge base lookup, and UI rendering).

High-level data flow:

User image → Validate (extension, size) → Load RGB PIL → Resize 224×224 → Normalise → Forward pass (ResNet50) → Softmax → Top-k classes → Disease DB lookup → Display results

4.2 Model: ResNet50 with Transfer Learning

ResNet50 is a 50-layer deep residual network. Each residual block computes:

$$y = F(x, \{W_i\}) + x$$

where x is the block input, $F(x, \{W_i\})$ is the residual mapping learned by the stacked weight layers, and the identity shortcut $+ x$ ensures that gradients flow directly through the network during backpropagation. ResNet50 uses bottleneck blocks, where a 3×3 convolution is flanked by 1×1 convolutions that compress and restore the channel dimension, reducing computational cost without sacrificing representational capacity.

The final fully-connected layer is replaced from $\text{Linear}(2048 \rightarrow 1000)$ to $\text{Linear}(2048 \rightarrow 38)$ to match the 38 PlantVillage classes. All convolutional weights are initialised from ImageNet pretraining.

4.3 Training Process

Backbone-Frozen Mode (default)

All convolutional parameters are frozen; only the $2048 \times 38 + 38 = 77,862$ parameters of the final FC layer are trained. This reduces the optimisation problem to a linear classifier on top of fixed ImageNet features and converges in a few epochs on CPU.

Full Fine-Tuning Mode (--full-finetune)

All 25.6 million parameters train simultaneously at a reduced learning rate ($\eta = 10^{-4}$) to avoid catastrophic forgetting of the pretrained representations. Recommended when a GPU is available.

Loss Function

Categorical cross-entropy is used as the training objective:

$$L = -(1/N) \sum_i \log p^{\hat{}}(i, y_i)$$

where N is the batch size, y_i is the true class index, and $p^{\hat{}}(i, c)$ is the predicted class probability obtained by applying softmax to the raw logits $z \in \mathbb{R}^{38}$:

$$\hat{p}(i,c) = \exp(z(i,c)) / \sum_j \exp(z(i,j))$$

Optimiser: Adam

Adam (Adaptive Moment Estimation) maintains exponential moving averages of the gradient (m) and its square (v), with bias correction:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\ \hat{m}_t &= m_t / (1-\beta_1^t), \quad \hat{v}_t = v_t / (1-\beta_2^t) \\ \theta_{t+1} &= \theta_t - (\alpha / (\sqrt{\hat{v}_t} + \epsilon)) \hat{m}_t \end{aligned}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 10^{-3}$ (frozen) / 10^{-4} (full fine-tune). The per-parameter adaptive step size is critical when fine-tuning, since different layer groups have very different gradient magnitudes.

Checkpointing

The model state dict is saved after each epoch that improves validation accuracy. Only the best checkpoint is retained, preventing overfitting from being baked into the saved weights.

5. Implementation Details

5.1 Technology Stack

Component	Technology	Version
Language	Python	3.9+
Deep learning framework	PyTorch	≥ 2.0.0
Model zoo / transforms	torchvision	≥ 0.15.0
Web interface	Streamlit	≥ 1.30.0
Image loading	Pillow (PIL)	≥ 10.0.0
Numerical operations	NumPy	≥ 1.24.0
Tabular data / CSV export	pandas	≥ 2.0.0
Evaluation metrics	scikit-learn	≥ 1.3.0
Confusion matrix visualisation	matplotlib	≥ 3.7.0
Dataset download	kaggle	≥ 1.6.0

5.2 Hardware

The model was trained and evaluated on a standard consumer laptop (CPU only). Device selection is automatic: the code queries for CUDA → Apple MPS → CPU in order of preference, so the same codebase runs unchanged on any hardware. Inference on CPU takes approximately 80–100 ms per image.

5.3 Code Structure

Module	Responsibility
src/model.py	ResNet50 definition; weight loading with state_dict / checkpoint support
src/preprocess.py	File validation, RGB loading, resize, and ImageNet normalisation
src/predict.py	Predictor class: model + class names + disease DB; CLI entry point
src/train.py	Training loop, data augmentation, checkpointing, class-name sync
src/evaluate.py	Full validation-set evaluation: classification report, confusion matrix
src/utils.py	Shared helpers: paths, device selection, label formatting
src/download_data.py	Kaggle dataset download with Windows long-path extraction
app/streamlit_app.py	Web UI: single-image and batch tabs, confidence badges

5.4 Key Engineering Decisions

- Windows MAX_PATH workaround: the Kaggle zip contains paths exceeding 260 characters. Extraction uses the \\?\ long-path prefix to bypass the Windows limit.

- Duplicate tree removal: the zip includes identical image trees under both upper-case and lower-case folder names. The extraction script detects and discards the duplicate.
- Checkpoint flexibility: the model loader accepts both a raw `state_dict` and a checkpoint dict with a `'state_dict'` key, ensuring compatibility with standard PyTorch training output.
- Streamlit caching: the Predictor is wrapped in `@st.cache_resource`, so the 90 MB model is loaded from disk only once per server session rather than on every upload.

6. Experimental Results

6.1 Overall Performance

The model was evaluated on the full PlantVillage validation split of 17,572 images using the backbone-frozen training mode (FC layer only). The following results were obtained:

Metric	Value
Overall accuracy	89.82%
Macro precision	90.21%
Macro recall	89.82%
Macro F1-score	89.51%
Weighted F1-score	89.53%
Samples evaluated	17,572

6.2 Per-Class Metrics

Per-class precision, recall, F1-score, and support are written to results/classification_report.csv by running:

```
python -m src.evaluate --data-dir data/raw
```

The macro and weighted F1 scores are close (89.51% vs 89.53%), confirming that the dataset is approximately class-balanced and the model does not achieve its headline accuracy by over-performing on dominant classes. The ~90% accuracy reflects backbone-frozen training; full fine-tuning is expected to push this above 95%.

6.3 Confusion Matrix Analysis

The row-normalised confusion matrix (results/confusion_matrix.png) shows recall per class: cell (i, j) gives the fraction of true-class-i samples predicted as class j, so a perfect model would be a pure diagonal. Reading the matrix reveals a clear structural pattern: the upper-left block (Apple through Strawberry) is almost entirely diagonal — most non-tomato species are classified with very high recall. The lower-right block, covering the ten Tomato disease classes, is visibly lighter and contains the majority of off-diagonal mass. This is not a coincidence — it is a direct consequence of what a frozen ImageNet backbone can and cannot represent.

The backbone encodes general visual features: leaf shape, colour, surface texture at a coarse scale. These features are sufficient to distinguish a grape leaf from a tomato leaf, or rust from powdery mildew across different host plants, because those differences are large and perceptible at the level of colour and gross morphology. What they cannot capture is the fine-grained intra-species variation between diseases that manifest as subtly different lesion textures on the same host plant. The ten

tomato disease classes all share the same leaf background — the discriminating signal lies entirely in lesion shape, ring structure, spot density, and colour hue, features that require fine-tuned domain-specific filters to separate.

Key Confusion Pairs

Tomato Target Spot (recall 36.3%, F1 0.52) is the single largest failure in the model. Target Spot produces circular lesions with concentric dark rings and a yellow halo — visually almost identical to Early Blight, which also shows concentric ring lesions caused by a different fungus (*Alternaria solani*). The confusion matrix shows the majority of Target Spot images being misclassified as Early Blight. To a frozen backbone, both diseases look like 'brown circular ring pattern on a green leaf', and distinguishing them requires recognising subtle differences in ring spacing, lesion border sharpness, and the degree of chlorotic yellowing — the kind of fine-grained discrimination that only emerges with domain-specific fine-tuning.

Tomato Early Blight (recall 69.0%, F1 0.73) and Tomato Late Blight (recall 70.6%, F1 0.71) are also heavily confused with each other and with Septoria Leaf Spot (F1 0.75). Early Blight creates dark brown spots with concentric rings and yellowing. Late Blight creates irregular water-soaked dark patches that spread rapidly. Septoria creates small circular spots with gray centres and dark margins. All three appear as 'dark spotted lesions on a green tomato leaf' to the backbone, making them the hardest cluster to separate without learning disease-specific spatial frequency patterns through fine-tuning.

Corn Cercospora Leaf Spot / Gray Leaf Spot (F1 0.87) is confused with Northern Leaf Blight (F1 0.89). Both diseases produce elongated rectangular lesions running parallel to the leaf veins on corn. Cercospora lesions are shorter and grayer; Northern Leaf Blight produces longer cigar-shaped tan lesions. Their shared elongated geometry and location on corn leaves makes them the primary cross-corn confusion pair.

Apple Scab (recall 80.2%, F1 0.88) is the worst-performing Apple class. Scab produces olive-green to dark velvety spots on the upper leaf surface. The confusion is primarily with Apple Healthy, suggesting that early-stage scab lesions — which are small and low-contrast — are not sufficiently distinct from a healthy leaf at the feature level encoded by the frozen backbone.

What the Confusion Pattern Tells Us

The confusion matrix reveals a two-stage implicit classification the model is performing: first, identify the host plant species (leaf shape, colour, macroscopic texture); second, identify the disease (lesion morphology, spot pattern, colour hue). The frozen backbone handles the first stage well — cross-species confusion is rare. It struggles at the second stage within species that have multiple visually similar diseases, particularly tomato. This is precisely the accuracy gap that full fine-tuning is

expected to close: end-to-end gradient flow would force the convolutional filters to learn lesion-discriminative features rather than reusing general ImageNet textures.

6.4 Baseline Comparison

Model / Approach	Val. Accuracy	Notes
Random baseline	2.6%	1/38 classes — chance performance
ResNet50 backbone-frozen (this project)	89.82%	Full validation set, 17,572 images; FC layer only
ResNet50 full fine-tune (expected)	~95–97%	All layers trained; GPU recommended
Mohanty et al. (2016) — GoogLeNet	98.90%	Same dataset; reported in original paper

The ~9% gap between backbone-frozen ResNet50 and the Mohanty et al. GoogLeNet result is explained by two factors: (1) their model was fully fine-tuned end-to-end, allowing all filters to specialise on disease textures, and (2) they used 20-fold cross-validation across the full dataset rather than the fixed Kaggle split used here. The confusion matrix shows the gap is concentrated almost entirely in the ten intra-tomato disease classes — on all other species the backbone-frozen model already achieves 90–99% recall.

7. Discussion

7.1 Strengths

- Strong accuracy (89.82% backbone-frozen, ~95–97% expected with full fine-tuning) across 38 fine-grained classes with a well-understood, reproducible architecture.
- Transfer learning makes the project practical on CPU: backbone-frozen training converges in a few epochs with only 77K trainable parameters.
- The disease knowledge base adds real value beyond classification — users receive concrete treatment and prevention guidance, not just a class label.
- Batch processing with CSV export enables integration into farm-management workflows.
- Device-agnostic design (CUDA / MPS / CPU) means the system deploys on any hardware.

7.2 Limitations

- Lab-only training data: PlantVillage images have controlled grey/black backgrounds. Performance on field photographs with complex backgrounds, variable lighting, and partial occlusion is expected to be substantially lower.
- Closed-class system: the model can only predict one of the 38 trained categories. It has no mechanism to detect or reject out-of-distribution inputs (e.g. a non-leaf image).
- No severity grading: the model classifies disease type but does not estimate severity (mild / moderate / severe), which is often the more useful clinical signal.
- Model size: at ~90 MB, the weights are large for browser or mobile deployment without quantisation or distillation.

7.3 Challenges Faced

- Windows MAX_PATH limit: the PlantVillage zip contains file paths exceeding 260 characters. Standard Python extraction fails silently. The solution uses the \\?\ UNC prefix for all extraction paths, which bypasses the Windows API limit.
- Duplicate dataset trees: the Kaggle zip contains two copies of the dataset under differently cased folder names. Custom extraction logic identifies and discards the duplicate to avoid class-label collision.
- CPU inference speed: ResNet50 processes ~10–12 images per second on a modern CPU. For batch jobs of hundreds of images, this is a bottleneck. The --num-workers DataLoader option partially mitigates this by parallelising preprocessing.

8. Conclusion and Future Work

8.1 Summary

This project shows that transfer learning from ImageNet onto a ResNet50 backbone produces a competitive plant disease classifier (89.82% backbone-frozen on 17,572 validation images) with practical training times on consumer hardware. The confusion matrix analysis reveals exactly where and why the model struggles: not across species, but within the tomato disease cluster where multiple conditions produce visually similar lesion patterns. The system is packaged as a deployable Streamlit web application with single-image and batch-processing modes, disease knowledge lookup, and CSV export. The codebase is modular and fully reproducible: dataset download, training, evaluation, and inference are each implemented as standalone command-line modules.

8.2 Future Improvements

- Full fine-tuning with GPU: the most direct improvement — end-to-end gradient flow forces convolutional filters to learn lesion-discriminative features, expected to push accuracy above 95% and close the tomato confusion cluster.
- Field-condition robustness: augment training with field-collected images using colour jitter, random cropping, Gaussian blur, and background replacement to close the lab-to-field gap.
- Out-of-distribution detection: add a confidence threshold below which predictions are rejected as 'unknown', preventing overconfident outputs on non-leaf inputs.
- Disease severity grading: extend the output to include a severity level (mild / moderate / severe) by training on annotated severity data or via weakly-supervised saliency maps.
- Model compression: apply post-training quantisation (INT8) or knowledge distillation to a MobileNetV3 student model to produce a version suitable for on-device mobile inference.
- Additional species: expand coverage to crops not in PlantVillage (e.g. rice, wheat, cassava) using domain adaptation from the existing pretrained features.

References

- [1] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419.
<https://doi.org/10.3389/fpls.2016.01419>
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [3] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1412.6980>
- [4] Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, 311–318.
<https://doi.org/10.1016/j.compag.2018.01.009>
- [5] Barbedo, J. G. A. (2016). A review on the main challenges in automatic plant disease identification based on visible range images. *Biosystems Engineering*, 144, 52–60.
<https://doi.org/10.1016/j.biosystemseng.2016.01.017>
- [6] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *Proceedings of CVPR*, pp. 248–255.
<https://doi.org/10.1109/CVPR.2009.5206848>
- [7] Vipooooool. (2020). New Plant Diseases Dataset [Data set]. Kaggle.
<https://www.kaggle.com/datasets/vipooooool/new-plant-diseases-dataset>

Appendix A — Supported Classes

All 38 classification targets, as stored in data/class_names.json:

#	Class Name	Display Label
1	Apple__Apple_scab	Apple - Apple scab
2	Apple__Black_rot	Apple - Black rot
3	Apple__Cedar_apple_rust	Apple - Cedar apple rust
4	Apple__healthy	Apple - Healthy
5	Blueberry__healthy	Blueberry - Healthy
6	Cherry_(including_sour)__Powdery_mildew	Cherry - Powdery mildew
7	Cherry_(including_sour)__healthy	Cherry - Healthy
8	Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot	Corn - Cercospora / Gray leaf spot
9	Corn_(maize)__Common_rust_	Corn - Common rust
10	Corn_(maize)__Northern_Leaf_Blight	Corn - Northern leaf blight
11	Corn_(maize)__healthy	Corn - Healthy
12	Grape__Black_rot	Grape - Black rot
13	Grape__Esca_(Black_Measles)	Grape - Esca (Black Measles)
14	Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	Grape - Leaf blight
15	Grape__healthy	Grape - Healthy
16	Orange__Haunglongbing_(Citrus_greening)	Orange - Huanglongbing
17	Peach__Bacterial_spot	Peach - Bacterial spot
18	Peach__healthy	Peach - Healthy
19	Pepper,_bell__Bacterial_spot	Bell Pepper - Bacterial spot
20	Pepper,_bell__healthy	Bell Pepper - Healthy
21	Potato__Early_blight	Potato - Early blight
22	Potato__Late_blight	Potato - Late blight
23	Potato__healthy	Potato - Healthy
24	Raspberry__healthy	Raspberry - Healthy
25	Soybean__healthy	Soybean - Healthy
26	Squash__Powdery_mildew	Squash - Powdery mildew
27	Strawberry__Leaf_scorch	Strawberry - Leaf scorch
28	Strawberry__healthy	Strawberry - Healthy
29	Tomato__Bacterial_spot	Tomato - Bacterial spot
30	Tomato__Early_blight	Tomato - Early blight
31	Tomato__Late_blight	Tomato - Late blight

32	Tomato__Leaf_Mold	Tomato - Leaf mold
33	Tomato__Septoria_leaf_spot	Tomato - Septoria leaf spot
34	Tomato__Spider_mites Two-spotted_spider_mite	Tomato - Spider mites
35	Tomato__Target_Spot	Tomato - Target spot
36	Tomato__Tomato_Yellow_Leaf_Curl_Virus	Tomato - Yellow leaf curl virus
37	Tomato__Tomato_mosaic_virus	Tomato - Mosaic virus
38	Tomato__healthy	Tomato - Healthy